

Universidade Federal de Viçosa

Campus

Rio Paranaíba

Leonardo Padovani Manzato – 9862

Análise de algoritmos de ordenação

Rio Paranaíba – MG

2026

Universidade Federal de Viçosa

Campus

Rio Paranaíba

Leonardo Padovani Manzato – 9862

Análise de algoritmos de ordenação

Trabalho apresentado para obtenção de créditos na disciplina SIN213 – Projeto de Algoritmo da Universidade Federal de Viçosa – Campus de Rio Paranaíba, ministrada pelo Professor Pedro Moisés de Souza.

Rio Paranaíba – MG

2026

1 RESUMO

Os algoritmos de ordenação apresentam grande importância no estudo teórico de algoritmos, bem como em aplicações práticas do cotidiano. Esses algoritmos são responsáveis por organizar conjuntos de elementos de acordo com determinado critério, facilitando a busca, a visualização e o processamento dos dados. Diante da existência de diferentes métodos de ordenação, torna-se importante compreender seu funcionamento e analisar seus custos para identificar suas características e limitações.

Neste trabalho, são analisados quatro algoritmos de ordenação: *Insertion Sort*, *Bubble Sort*, *Selection Sort* e *Shell Sort*. Cada algoritmo apresenta uma estratégia diferente para organizar os elementos de um vetor. Enquanto o *Insertion Sort* realiza inserções progressivas, o *Bubble Sort* utiliza trocas entre elementos adjacentes, o *Selection Sort* seleciona sucessivamente o menor elemento e o *Shell Sort* utiliza intervalos (*gaps*) para realizar ordenações parciais antes da ordenação final.

Para cada algoritmo, são apresentados sua implementação, um teste de mesa manual e uma análise de complexidade, considerando os diferentes casos de execução. Também são realizados experimentos com entradas ordenadas de forma crescente, decrescente e aleatória, utilizando diferentes tamanhos de instância. Os tempos de execução obtidos são apresentados por meio de tabelas e gráficos, permitindo relacionar o comportamento observado experimentalmente com a análise teórica de cada algoritmo.

Dessa forma, o trabalho busca compreender as diferenças entre os algoritmos estudados e analisar como suas características influenciam o desempenho durante a ordenação de diferentes tipos e tamanhos de entrada.

Sumário

1	RESUMO	1
2	INTRODUÇÃO	4
3	ALGORITMOS	5
3.1	Insertion Sort	5
3.2	Bubble Sort	5
3.3	Selection Sort	7
3.4	Shell Sort	8
4	ANÁLISE DE COMPLEXIDADE	10
4.1	Insertion Sort	10
4.1.1	Melhor caso – $\Theta(n)$	10
4.1.2	Pior caso – $\Theta(n^2)$	10
4.1.3	Caso médio – $\Theta(n^2)$	11
4.1.4	Resumo das complexidades	12
4.2	Bubble Sort	12
4.2.1	Melhor caso – $\Theta(n^2)$	12
4.2.2	Pior caso – $\Theta(n^2)$	13
4.2.3	Caso médio – $\Theta(n^2)$	13
4.2.4	Resumo das complexidades	14
4.3	Selection Sort	14
4.3.1	Melhor caso – $\Theta(n^2)$	15
4.3.2	Pior caso – $\Theta(n^2)$	15
4.3.3	Caso médio – $\Theta(n^2)$	16
4.3.4	Resumo das complexidades	17
4.4	Shell Sort	17
4.4.1	Melhor caso – $\Theta(n \log n)$	18
4.4.2	Pior caso – $\Theta(n^2)$	18
4.4.3	Caso médio	19
4.4.4	Resumo das complexidades	20

5	TABELAS E GRÁFICOS	21
5.1	Insertion Sort	21
5.2	Bubble Sort	22
5.3	Selection Sort	24
5.4	Shell Sort	25
6	COMPARAÇÃO ENTRE OS ALGORITMOS	28
6.1	Entrada crescente	28
6.2	Entrada decrescente	28
6.3	Entrada aleatória	29
7	ATIVIDADES MANUSCRITAS	32
7.1	Insertion Sort	32
7.2	Bubble Sort	34
7.3	Selection Sort	36
7.4	Shell Sort	38
8	CONCLUSÃO	40
9	REFERÊNCIAS BIBLIOGRÁFICAS	42
10	FUNÇÃO PARA CALCULAR O TEMPO	44

2 INTRODUÇÃO

Em várias situações do nosso dia a dia, nos deparamos com a necessidade de trabalhar com dados e informações ordenados de forma correta, como, por exemplo, ao procurar um contato na lista telefônica. Seria difícil localizar um nome caso os contatos não estivessem organizados em ordem alfabética. Dessa forma, fica fácil notar que as atividades que envolvem algum método de ordenação estão muito presentes na computação.

Os algoritmos de ordenação são responsáveis por organizar um conjunto de elementos de acordo com determinado critério, como ordem crescente, decrescente ou alfabética. Essa organização facilita a busca, a visualização e o processamento dos dados, contribuindo para a eficiência de diversas aplicações.

Existem diferentes algoritmos de ordenação, cada um apresentando características, estratégias e custos computacionais próprios. Entre eles, destacam-se os algoritmos *Insertion Sort*, *Bubble Sort*, *Selection Sort* e *Shell Sort*. Apesar de possuírem o mesmo objetivo, esses algoritmos utilizam diferentes métodos para organizar os elementos de um vetor.

O *Insertion Sort*, também conhecido como ordenação por inserção, constrói progressivamente uma parte ordenada do vetor, inserindo cada novo elemento em sua posição correta. O *Bubble Sort* realiza comparações entre elementos adjacentes e troca suas posições quando estão fora de ordem. Já o *Selection Sort* procura o menor elemento da parte ainda não ordenada e o posiciona na posição correspondente. Por sua vez, o *Shell Sort* é uma variação do método de inserção que realiza ordenações parciais utilizando intervalos, chamados de *gaps*, que são reduzidos progressivamente até que todo o vetor seja ordenado.

A análise da complexidade desses algoritmos é importante para compreender como o tempo de execução varia de acordo com o tamanho e a organização inicial da entrada. Dependendo do algoritmo e do caso analisado, podem existir diferenças significativas entre o melhor caso, o caso médio e o pior caso. No *Shell Sort*, além do tamanho da entrada, o comportamento também está relacionado à sequência de *gaps* utilizada.

Neste trabalho, serão apresentados o funcionamento e a implementação dos algoritmos *Insertion Sort*, *Bubble Sort*, *Selection Sort* e *Shell Sort*, juntamente com testes de mesa e análises de complexidade. Também serão realizados experimentos utilizando entradas crescentes, decrescentes e aleatórias, considerando diferentes tamanhos de instância.

3 ALGORITMOS

3.1 Insertion Sort

O Insertion Sort é um algoritmo de ordenação baseado na comparação e inserção dos elementos em suas posições corretas. Seu funcionamento é semelhante ao ato de organizar cartas de baralho: os elementos são analisados um por vez e cada novo elemento é inserido na posição adequada da parte que já está ordenada.

Inicialmente, considera-se que o primeiro elemento do vetor já está ordenado. Em seguida, o algoritmo seleciona o próximo elemento, chamado de chave, e compara seu valor com os elementos anteriores. Enquanto esses elementos forem maiores que a chave, eles são deslocados uma posição para a direita. Ao final, a chave é inserida na posição correta.

Listing 1: Implementação do Insertion Sort

```
1 void insertionSort(int* vetor, int tamanho) {  
2     for (int i = 1; i < tamanho; i++) {  
3  
4         int chave = vetor[i];  
5         int j = i - 1;  
6  
7         while (j >= 0 && vetor[j] > chave) {  
8             vetor[j + 1] = vetor[j];  
9             j--;  
10        }  
11  
12        vetor[j + 1] = chave;  
13    }  
14 }
```

3.2 Bubble Sort

O Bubble Sort é um algoritmo de ordenação baseado na comparação e troca de elementos adjacentes. Seu funcionamento consiste em percorrer o vetor várias vezes, comparando elementos vizinhos e realizando uma troca quando eles estão fora da ordem desejada.

Dessa forma, os maiores elementos vão sendo deslocados gradualmente para o final do vetor, de maneira semelhante ao movimento de bolhas subindo em um líquido.

A cada passagem pelo vetor, os elementos adjacentes são comparados. Caso o elemento da esquerda seja maior que o elemento da direita, suas posições são trocadas. Após cada passagem, pelo menos um dos maiores elementos ainda não ordenados é colocado em sua posição definitiva no final do vetor. O processo continua até que todos os elementos estejam ordenados.

Listing 2: Implementação do Bubble Sort

```
1 void bubbleSort(int* vetor, int tamanho) {
2     for (int i = 0; i < tamanho - 1; i++) {
3         bool trocou = false;
4
5         for (int j = 0; j < tamanho - i - 1; j++) {
6             if (vetor[j] > vetor[j + 1]) {
7                 int temp = vetor[j];
8                 vetor[j] = vetor[j + 1];
9                 vetor[j + 1] = temp;
10                trocou = true;
11            }
12        }
13        if (!trocou)
14            break;
15    }
16 }
```

No primeiro laço, a variável *i* representa a quantidade de passagens já realizadas pelo vetor. O segundo laço percorre os elementos ainda não ordenados, comparando cada elemento com seu vizinho à direita.

Quando o elemento da esquerda é maior que o elemento da direita, é realizada uma troca entre eles utilizando a variável auxiliar *temp*. Ao final de cada passagem, o maior elemento da parte ainda não ordenada é levado para o final do vetor. Por isso, o limite do segundo laço diminui a cada nova passagem.

3.3 Selection Sort

O Selection Sort é um algoritmo de ordenação baseado na seleção sucessiva do menor elemento do vetor. Seu funcionamento consiste em dividir mentalmente o vetor em duas partes: uma parte inicialmente desordenada e uma parte que já foi ordenada. A cada iteração, o algoritmo procura o menor elemento da parte desordenada e o coloca na posição correspondente da parte ordenada.

Inicialmente, considera-se que nenhum elemento está ordenado. Durante cada iteração, o algoritmo percorre os elementos restantes do vetor e identifica aquele que possui o menor valor. Após encontrar esse elemento, ele é trocado com o primeiro elemento da parte ainda desordenada. Dessa forma, a parte ordenada cresce progressivamente até que todos os elementos estejam em suas posições corretas.

Listing 3: Implementação do Selection Sort

```
1 void selectionSort(int* vetor, int tamanho) {  
2     for (int i = 0; i < tamanho - 1; i++) {  
3         int menor = i;  
4  
5         for (int j = i + 1; j < tamanho; j++) {  
6             if (vetor[j] < vetor[menor]) {  
7                 menor = j;  
8             }  
9         }  
10  
11         int temp = vetor[i];  
12         vetor[i] = vetor[menor];  
13         vetor[menor] = temp;  
14     }  
15 }
```

No primeiro laço, a variável *i* representa o início da parte ainda não ordenada do vetor. A variável **menor** armazena a posição do menor elemento encontrado durante a busca.

O segundo laço percorre os elementos que ainda não foram ordenados, comparando seus valores com o elemento armazenado na posição **menor**. Quando um elemento menor

é encontrado, sua posição passa a ser armazenada nessa variável.

Ao final de cada iteração, o menor elemento encontrado é trocado com o elemento que ocupa a posição i . Assim, esse elemento passa a fazer parte da região ordenada do vetor. O processo é repetido até que restem apenas dois elementos ou até que todo o vetor esteja ordenado.

3.4 Shell Sort

O Shell Sort é um algoritmo de ordenação baseado no Insertion Sort, porém utiliza comparações entre elementos que estão separados por determinados intervalos, chamados de *gaps*. Essa estratégia permite reduzir gradualmente a desordem do vetor antes da etapa final de ordenação.

Inicialmente, é definido um valor de *gap* que determina a distância entre os elementos que serão comparados. O vetor é então percorrido considerando essa distância, realizando uma ordenação por inserção entre os elementos pertencentes aos diferentes grupos formados. Após cada etapa, o valor do *gap* é reduzido utilizando um divisor, fazendo com que os elementos sejam comparados a distâncias cada vez menores.

O processo continua até que o *gap* seja igual a 1. Nesse momento, é realizada uma última ordenação semelhante ao Insertion Sort, porém o vetor já se encontra parcialmente ordenado devido às etapas anteriores, reduzindo bastante a quantidade de deslocamentos necessários.

Listing 4: Implementação do Shell Sort

```
1 void shellSort(int* vetor, int tamanho, int divisor) {  
2  
3     for (int gap = tamanho / divisor; gap > 0; gap /= divisor) {  
4         for (int i = gap; i < tamanho; i++) {  
5  
6             int temp = vetor[i];  
7  
8             int j = i;  
9  
10            while (j >= gap && vetor[j - gap] > temp) {  
11
```

```
12         vetor[j] = vetor[j - gap];
13         j -= gap;
14     }
15
16     vetor[j] = temp;
17 }
18 }
19 }
```

Na primeira parte do algoritmo, o valor inicial do *gap* é obtido pela divisão do tamanho do vetor pelo divisor informado. A cada nova etapa, o *gap* é novamente dividido pelo mesmo divisor. Dessa forma, o intervalo entre os elementos comparados diminui progressivamente.

O segundo laço percorre o vetor a partir da posição correspondente ao *gap*. Para cada elemento, a variável **temp** armazena seu valor, enquanto o terceiro laço desloca os elementos maiores que **temp** para a posição seguinte dentro do intervalo definido pelo *gap*.

Quando não existem mais elementos maiores que **temp** na sequência considerada, o valor é inserido em sua posição correta. Quando o *gap* chega a 1, o algoritmo realiza uma ordenação por inserção considerando elementos consecutivos. Como as etapas anteriores já reduziram parte da desordem do vetor, essa última etapa tende a exigir menos deslocamentos.

Inicialmente, pretendia-se permitir a alteração do divisor utilizado na redução dos *gaps* durante a execução do algoritmo, com o objetivo de comparar o desempenho do Shell Sort utilizando diferentes sequências de *gaps*. Entretanto, devido a dificuldades na implementação dessa alteração, foi adotado o divisor padrão igual a 2 para os experimentos realizados neste trabalho. Dessa forma, a sequência de *gaps* é obtida pela divisão sucessiva do tamanho do vetor por 2 até que o *gap* seja igual a 1.

4 ANÁLISE DE COMPLEXIDADE

4.1 Insertion Sort

O algoritmo Insertion Sort apresenta diferentes comportamentos de desempenho dependendo da organização inicial dos elementos do vetor. Sua complexidade de tempo é analisada considerando o melhor caso, o caso médio e o pior caso.

4.1.1 Melhor caso – $\Theta(n)$

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Nessa situação, para cada elemento analisado, o algoritmo realiza apenas uma comparação com o elemento anterior e verifica que não é necessário deslocar nenhum elemento.

Considerando um vetor com n elementos, o laço externo é executado $n - 1$ vezes. Em cada iteração, é realizada uma quantidade constante de operações, como atribuições e comparações.

Assim, o número total de operações é proporcional a:

$$T(n) = c(n - 1) + d \quad (1)$$

em que c e d são constantes. Desconsiderando os termos constantes e os de menor crescimento, temos:

$$T(n) = \Theta(n) \quad (2)$$

Portanto, o melhor caso do Insertion Sort possui complexidade linear.

4.1.2 Pior caso – $\Theta(n^2)$

O pior caso ocorre quando o vetor está ordenado de forma decrescente e deseja-se ordená-lo de forma crescente. Nesse cenário, cada novo elemento precisa ser comparado com todos os elementos anteriores e deslocado até o início da parte já ordenada.

A quantidade total de deslocamentos pode ser representada pela soma:

$$T(n) = 1 + 2 + 3 + \cdots + (n - 1) \quad (3)$$

Essa soma corresponde à soma dos termos de uma progressão aritmética:

$$T(n) = \frac{n(n-1)}{2} \quad (4)$$

Desenvolvendo a expressão:

$$T(n) = \frac{n^2 - n}{2} \quad (5)$$

O termo de maior crescimento é n^2 . Portanto:

$$T(n) = \Theta(n^2) \quad (6)$$

Assim, o pior caso do Insertion Sort apresenta complexidade quadrática.

4.1.3 Caso médio – $\Theta(n^2)$

O caso médio ocorre quando os elementos estão distribuídos aleatoriamente. Nessa situação, o algoritmo realiza, em média, aproximadamente metade dos deslocamentos possíveis para cada elemento inserido.

Temos:

$$T(n) \approx \frac{1}{2} (1 + 2 + 3 + \dots + (n-1)) \quad (7)$$

Aplicando a fórmula da soma:

$$T(n) \approx \frac{1}{2} \cdot \frac{n(n-1)}{2} \quad (8)$$

$$T(n) \approx \frac{n^2 - n}{4} \quad (9)$$

Como o termo dominante é n^2 , a complexidade assintótica é:

$$T(n) = \Theta(n^2) \quad (10)$$

Portanto, no caso médio, o Insertion Sort também apresenta complexidade quadrática.

4.1.4 Resumo das complexidades

Tabela 1: Complexidades do algoritmo Insertion Sort

Caso	Complexidade de tempo
Melhor caso	$\Theta(n)$
Caso médio	$\Theta(n^2)$
Pior caso	$\Theta(n^2)$

Conclui-se que o Insertion Sort é eficiente para vetores pequenos ou parcialmente ordenados, mas apresenta desempenho inferior para grandes conjuntos de dados desordenados.

4.2 Bubble Sort

O algoritmo Bubble Sort apresenta comportamento semelhante nos diferentes tipos de entrada, pois realiza comparações entre elementos adjacentes durante sucessivas passagens pelo vetor. Sua complexidade de tempo é analisada considerando o melhor caso, o caso médio e o pior caso.

4.2.1 Melhor caso – $\Theta(n^2)$

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Mesmo nessa situação, a implementação utilizada continua realizando todas as comparações previstas pelos laços do algoritmo, embora nenhuma troca seja necessária.

Considerando um vetor com n elementos, na primeira passagem são realizadas $n - 1$ comparações. Na segunda passagem, são realizadas $n - 2$ comparações, e assim sucessivamente até a última passagem, que realiza apenas uma comparação.

Assim, o número total de comparações pode ser representado pela soma:

$$T(n) = (n - 1) + (n - 2) + \cdots + 2 + 1 \quad (11)$$

Essa soma corresponde à soma dos primeiros $n - 1$ números naturais:

$$T(n) = \frac{n(n - 1)}{2} \quad (12)$$

Desenvolvendo a expressão:

$$T(n) = \frac{n^2 - n}{2} \quad (13)$$

O termo de maior crescimento é n^2 . Portanto:

$$T(n) = \Theta(n^2) \quad (14)$$

Assim, o melhor caso do Bubble Sort apresenta complexidade quadrática.

4.2.2 Pior caso – $\Theta(n^2)$

O pior caso ocorre quando o vetor está ordenado de forma decrescente e deseja-se ordená-lo de forma crescente. Nesse cenário, além de realizar todas as comparações previstas pelos laços, o algoritmo também precisa realizar uma grande quantidade de trocas entre os elementos.

A quantidade de comparações realizadas permanece igual à do melhor caso, sendo representada por:

$$T(n) = (n - 1) + (n - 2) + \cdots + 2 + 1 \quad (15)$$

Aplicando a fórmula da soma:

$$T(n) = \frac{n(n - 1)}{2} \quad (16)$$

$$T(n) = \frac{n^2 - n}{2} \quad (17)$$

Além das comparações, são realizadas trocas dos elementos que estão fora de ordem. No pior caso, a quantidade de trocas também cresce quadraticamente.

Como o termo dominante é n^2 , temos:

$$T(n) = \Theta(n^2) \quad (18)$$

Portanto, o pior caso do Bubble Sort apresenta complexidade quadrática.

4.2.3 Caso médio – $\Theta(n^2)$

O caso médio ocorre quando os elementos estão distribuídos de maneira aleatória. Nessa situação, o algoritmo continua realizando todas as passagens e comparações previstas pelos seus laços.

A quantidade de comparações é dada por:

$$T(n) = (n - 1) + (n - 2) + \cdots + 2 + 1 \quad (19)$$

Aplicando a fórmula da soma:

$$T(n) = \frac{n(n - 1)}{2} \quad (20)$$

$$T(n) = \frac{n^2 - n}{2} \quad (21)$$

Embora a quantidade de trocas varie de acordo com a organização inicial dos elementos, o número de comparações continua sendo proporcional a n^2 .

Como o termo dominante é n^2 , a complexidade assintótica é:

$$T(n) = \Theta(n^2) \quad (22)$$

Portanto, no caso médio, o Bubble Sort também apresenta complexidade quadrática.

4.2.4 Resumo das complexidades

Tabela 2: Complexidades do algoritmo Bubble Sort

Caso	Complexidade de tempo
Melhor caso	$\Theta(n)$
Caso médio	$\Theta(n^2)$
Pior caso	$\Theta(n^2)$

Conclui-se que o Bubble Sort apresenta complexidade quadrática nos três casos analisados. Mesmo quando o vetor já está ordenado, a implementação utilizada continua realizando as comparações das passagens previstas. Por esse motivo, o algoritmo pode apresentar desempenho limitado para grandes conjuntos de dados.

4.3 Selection Sort

O algoritmo Selection Sort apresenta comportamento semelhante nos diferentes tipos de entrada, pois a quantidade de comparações realizadas não depende da organização inicial

dos elementos do vetor. Sua complexidade de tempo é analisada considerando o melhor caso, o caso médio e o pior caso.

4.3.1 Melhor caso – $\Theta(n^2)$

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Mesmo nessa situação, o algoritmo precisa percorrer a parte ainda não ordenada do vetor para verificar qual é o menor elemento.

Considerando um vetor com n elementos, na primeira iteração são realizadas $n - 1$ comparações. Na segunda iteração, são realizadas $n - 2$ comparações, e assim sucessivamente até a última iteração, que realiza apenas uma comparação.

A quantidade total de comparações pode ser representada pela soma:

$$T(n) = (n - 1) + (n - 2) + \cdots + 2 + 1 \quad (23)$$

Essa soma corresponde à soma dos primeiros $n - 1$ números naturais:

$$T(n) = \frac{n(n - 1)}{2} \quad (24)$$

Desenvolvendo a expressão:

$$T(n) = \frac{n^2 - n}{2} \quad (25)$$

O termo de maior crescimento é n^2 . Portanto:

$$T(n) = \Theta(n^2) \quad (26)$$

Assim, o melhor caso do Selection Sort apresenta complexidade quadrática.

4.3.2 Pior caso – $\Theta(n^2)$

O pior caso ocorre quando o vetor está organizado de forma que seja necessário realizar várias trocas para colocá-lo em ordem crescente. Entretanto, mesmo nesse cenário, a quantidade de comparações realizadas pelo algoritmo permanece a mesma.

Na primeira iteração, são realizadas $n - 1$ comparações; na segunda, $n - 2$ comparações, e assim sucessivamente. Dessa forma:

$$T(n) = (n - 1) + (n - 2) + \cdots + 2 + 1 \quad (27)$$

Aplicando a fórmula da soma:

$$T(n) = \frac{n(n - 1)}{2} \quad (28)$$

$$T(n) = \frac{n^2 - n}{2} \quad (29)$$

Como o termo dominante é n^2 , temos:

$$T(n) = \Theta(n^2) \quad (30)$$

Portanto, o pior caso do Selection Sort apresenta complexidade quadrática.

4.3.3 Caso médio – $\Theta(n^2)$

O caso médio ocorre quando os elementos estão distribuídos de maneira aleatória. Nessa situação, assim como nos demais casos, o algoritmo precisa percorrer toda a parte ainda não ordenada do vetor para encontrar o menor elemento.

A quantidade de comparações continua sendo representada por:

$$T(n) = (n - 1) + (n - 2) + \cdots + 2 + 1 \quad (31)$$

Aplicando a fórmula da soma:

$$T(n) = \frac{n(n - 1)}{2} \quad (32)$$

$$T(n) = \frac{n^2 - n}{2} \quad (33)$$

Como o termo dominante é n^2 , a complexidade assintótica é:

$$T(n) = \Theta(n^2) \quad (34)$$

Portanto, no caso médio, o Selection Sort também apresenta complexidade quadrática.

4.3.4 Resumo das complexidades

Tabela 3: Complexidades do algoritmo Selection Sort

Caso	Complexidade de tempo
Melhor caso	$\Theta(n^2)$
Caso médio	$\Theta(n^2)$
Pior caso	$\Theta(n^2)$

Conclui-se que o Selection Sort apresenta complexidade quadrática nos três casos analisados. Diferentemente do Insertion Sort, a organização inicial do vetor não reduz significativamente a quantidade de comparações, pois o algoritmo precisa procurar o menor elemento em toda a parte ainda não ordenada a cada iteração.

4.4 Shell Sort

O algoritmo Shell Sort apresenta um comportamento diferente dos demais algoritmos analisados, pois sua complexidade depende da sequência de *gaps* utilizada. Neste trabalho, o *gap* inicial é obtido pela divisão do tamanho do vetor pelo divisor escolhido e, a cada etapa, é novamente dividido pelo mesmo divisor até atingir o valor 1.

Na implementação utilizada nos experimentos, foi adotado o divisor igual a 2. Dessa forma, para um vetor de tamanho n , a sequência de *gaps* é aproximadamente:

$$\frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1 \quad (35)$$

Caso outro divisor seja utilizado, a sequência de *gaps* também será alterada. Por exemplo, para um vetor de tamanho 64, utilizando divisor igual a 2, os *gaps* serão:

$$32, 16, 8, 4, 2, 1 \quad (36)$$

Já utilizando divisor igual a 4, teremos:

$$16, 4, 1 \quad (37)$$

Assim, um divisor maior reduz a quantidade de etapas realizadas. Porém, isso não significa necessariamente que o algoritmo será mais rápido, pois a sequência de *gaps* também determina a quantidade de desordem eliminada antes da etapa final.

4.4.1 Melhor caso – $\Theta(n \log n)$

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Nessa situação, em cada etapa do algoritmo, os elementos já estão na ordem correta para o *gap* utilizado. Dessa forma, o algoritmo precisa percorrer os elementos do vetor, mas não realiza deslocamentos significativos.

Em cada etapa, o laço responsável pela ordenação percorre aproximadamente n elementos. Quando o divisor utilizado é constante, como no caso do divisor 2 adotado neste trabalho, o número de etapas é proporcional a $\log n$.

Assim, o custo pode ser representado aproximadamente por:

$$T(n) = n \log n \quad (38)$$

Portanto:

$$T(n) = \Theta(n \log n) \quad (39)$$

Assim, para uma sequência de *gaps* obtida pela divisão sucessiva por uma constante, o melhor caso apresenta crescimento de ordem $n \log n$.

4.4.2 Pior caso – $\Theta(n^2)$

No pior caso, a organização inicial dos elementos pode exigir uma grande quantidade de comparações e deslocamentos durante as etapas do algoritmo. O maior custo pode ocorrer na etapa final, quando o *gap* é igual a 1.

Com *gap* igual a 1, o Shell Sort realiza uma ordenação semelhante ao Insertion Sort sobre o vetor inteiro. No pior caso, cada elemento pode precisar ser comparado e deslocado em relação aos elementos anteriores.

A quantidade de operações pode ser representada por:

$$T(n) = 1 + 2 + 3 + \cdots + (n - 1) \quad (40)$$

Aplicando a fórmula da soma:

$$T(n) = \frac{n(n - 1)}{2} \quad (41)$$

Desenvolvendo a expressão:

$$T(n) = \frac{n^2 - n}{2} \quad (42)$$

O termo de maior crescimento é n^2 . Portanto:

$$T(n) = \Theta(n^2) \quad (43)$$

Assim, considerando a sequência de *gaps* utilizada neste trabalho, o pior caso apresenta complexidade quadrática.

4.4.3 Caso médio

O caso médio ocorre quando os elementos estão distribuídos de maneira aleatória. Diferentemente de algoritmos como Insertion Sort, Bubble Sort e Selection Sort, não é possível determinar uma única complexidade média do Shell Sort apenas considerando o tamanho n do vetor.

Isso ocorre porque o comportamento do algoritmo depende diretamente da sequência de *gaps* utilizada. Alterando o divisor, diferentes sequências de *gaps* são produzidas, modificando a quantidade de etapas e a quantidade de elementos que são comparados e deslocados em cada etapa.

Por exemplo, considerando um vetor de tamanho 64, o divisor 2 produz a sequência:

$$32, 16, 8, 4, 2, 1 \quad (44)$$

Enquanto o divisor 4 produz:

$$16, 4, 1 \quad (45)$$

Apesar de o divisor 4 produzir menos etapas, os elementos permanecem mais tempo separados por intervalos maiores entre uma etapa e outra. Consequentemente, a quantidade de comparações e deslocamentos realizada na etapa final pode ser diferente.

Dessa forma, a complexidade média do Shell Sort deve ser analisada de acordo com a sequência de *gaps* adotada e com o modelo de entrada. Neste trabalho, não será atribuída uma complexidade assintótica ao caso médio, pois o divisor utilizado pode modificar o

comportamento do algoritmo.

4.4.4 Resumo das complexidades

Tabela 4: Complexidades do algoritmo Selection Sort

Caso	Complexidade de tempo
Melhor caso	$\Theta(n \log n)$
Caso médio	Depende do intervalo das comparações
Pior caso	$\Theta(n^2)$

Conclui-se que o Shell Sort possui uma complexidade que depende da sequência de *gaps* utilizada. O divisor determina como os *gaps* são reduzidos e, conseqüentemente, influencia a quantidade de etapas realizadas pelo algoritmo. Divisores diferentes produzem sequências diferentes e podem apresentar comportamentos distintos durante a execução.

Neste trabalho, foi utilizado o divisor 2 nos experimentos. Essa escolha produz uma sequência de *gaps* obtida pela divisão sucessiva por 2, até atingir o valor 1. A comparação entre diferentes divisores seria uma possibilidade para analisar experimentalmente como a escolha da sequência de *gaps* influencia o tempo de execução do Shell Sort.

5 TABELAS E GRÁFICOS

5.1 Insertion Sort

Resultados dos experimentos — Insertion Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	0.0	0.0	0.0
100.0	0.0	3e-06	3e-06
1000.0	1e-06	0.000239	0.000166
10000.0	7e-06	0.048499	0.0233
100000.0	6.8e-05	4.964015	2.45468
1000000.0	0.001696	535.4135	272.5905

Figura 1: Tabela de tempo de execução do algoritmo Insertion Sort

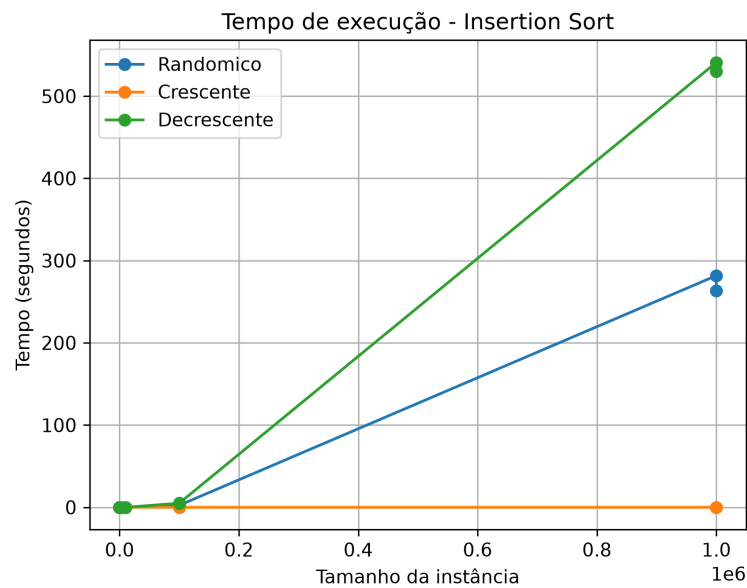


Figura 2: Gráfico de tempo de execução do algoritmo Insertion Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. Observa-se que o algoritmo Insertion Sort apresenta melhor desempenho para instâncias ordenadas de forma crescente, uma vez que, nesse caso, praticamente não são necessárias movimentações dos elementos.

Em contrapartida, as instâncias ordenadas de forma decrescente apresentam os maiores tempos de execução. Isso ocorre porque cada novo elemento precisa ser comparado e deslocado por uma grande quantidade de posições, caracterizando o pior caso do algoritmo, cuja complexidade é $\Theta(n^2)$.

As instâncias aleatórias apresentam desempenho intermediário, pois a quantidade de deslocamentos depende da disposição inicial dos elementos.

A partir dos dados, também é possível perceber que o aumento do tamanho da entrada provoca um crescimento significativo no tempo de execução, principalmente nos casos aleatório e decrescente. Para a instância de tamanho $n = 1.000.000$, o tempo registrado foi aproximadamente 272,59 segundos para a entrada aleatória e 535,41 segundos para a entrada decrescente. Já para a entrada crescente, o tempo foi de aproximadamente 0,001696 segundos.

Esses resultados confirmam o comportamento teórico esperado para o algoritmo Insertion Sort.

5.2 Bubble Sort

Resultados dos experimentos — Bubble Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	0.0	0.0	0.0
100.0	0.0	1.4e-05	3e-05
1000.0	1e-06	0.000962	0.001032
10000.0	8e-06	0.106261	0.094531
100000.0	0.0001	10.2698	17.7803
1000000.0	0.000797	1156.37	1954.39

Figura 3: Tabela de tempo de execução do algoritmo Bubble Sort

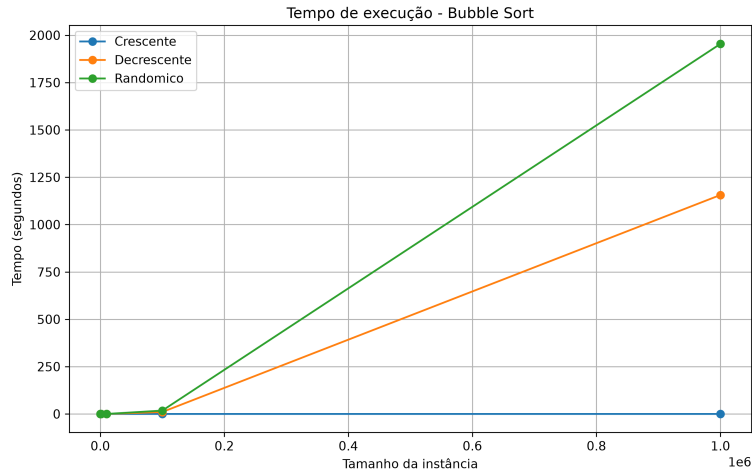


Figura 4: Gráfico de tempo de execução do algoritmo Bubble Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. Observa-se que o algoritmo Bubble Sort apresenta comportamentos diferentes de acordo com a organização inicial dos elementos, devido à utilização da variável `trocou`, que permite interromper a execução quando nenhuma troca é realizada durante uma passagem.

Nas instâncias ordenadas de forma crescente, nenhuma troca é realizada durante a primeira passagem. Dessa forma, a variável `trocou` permanece com valor `false` e o algoritmo encerra sua execução por meio do comando `break`, fazendo com que o melhor caso apresente complexidade $\Theta(n)$.

Nas instâncias ordenadas de forma decrescente, além das comparações, também são necessárias diversas trocas para colocar os elementos em suas posições corretas. Como ocorre pelo menos uma troca durante cada passagem, o comando `break` não é executado antes do término das passagens previstas, caracterizando o pior caso do algoritmo, cuja complexidade é $\Theta(n^2)$.

As instâncias aleatórias apresentam uma quantidade intermediária de trocas, pois o número de elementos que precisam ser deslocados depende da organização inicial do vetor. Entretanto, o algoritmo normalmente realiza diversas passagens pelo vetor, fazendo com que o caso médio apresente complexidade $\Theta(n^2)$.

A partir dos dados, também é possível perceber que o aumento do tamanho da entrada provoca um crescimento significativo no tempo de execução, principalmente nas entradas aleatórias e decrescentes. Nas entradas crescentes, o algoritmo pode encerrar a execução após a primeira passagem, devido à ausência de trocas.

Esses resultados apresentam comportamento compatível com a análise de complexi-

dade do algoritmo Bubble Sort, considerando a otimização utilizada na implementação por meio da variável `trocou`.

5.3 Selection Sort

Resultados dos experimentos — Selection Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	0.0	1e-06	1e-06
100.0	8e-06	1.1e-05	1e-05
1000.0	0.000397	0.000403	0.00041
10000.0	0.035705	0.036678	0.035624
100000.0	3.77752	3.91593	3.70861
1000000.0	403.486	412.212	392.715

Figura 5: Tabela de tempo de execução do algoritmo Selection Sort

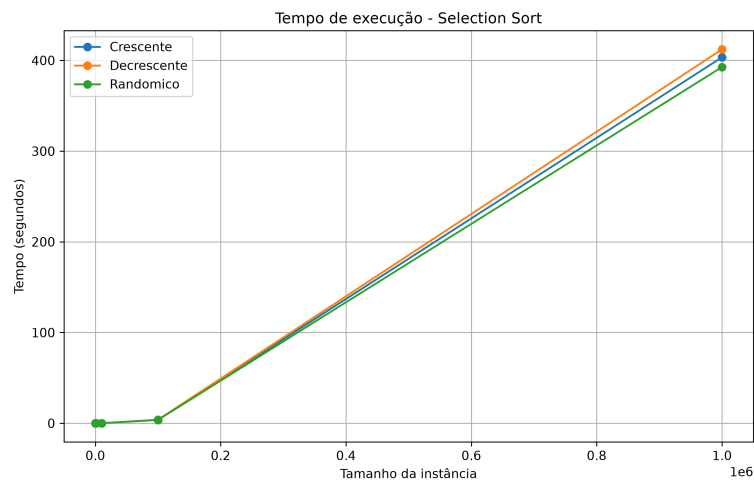


Figura 6: Gráfico de tempo de execução do algoritmo Selection Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. Observa-se que o algoritmo Selection Sort apresenta comportamento semelhante nos diferentes tipos de entrada, pois a quantidade de comparações realizadas não depende da organização inicial dos elementos.

Nas instâncias ordenadas de forma crescente, o algoritmo ainda precisa percorrer a parte não ordenada do vetor para encontrar o menor elemento. Dessa forma, mesmo

quando os elementos já estão organizados, são realizadas aproximadamente as mesmas comparações previstas para os demais tipos de entrada.

Nas instâncias ordenadas de forma decrescente, o algoritmo também realiza a mesma quantidade de comparações, embora a posição do menor elemento encontrado possa exigir mais trocas para colocá-lo na posição correta.

As instâncias aleatórias apresentam comportamento semelhante, pois, independentemente da disposição inicial dos elementos, o algoritmo continua procurando o menor elemento em toda a parte ainda não ordenada do vetor.

A partir dos dados, também é possível perceber que o aumento do tamanho da entrada provoca um crescimento significativo no tempo de execução. Esse comportamento está de acordo com a análise teórica, segundo a qual o Selection Sort apresenta complexidade $\Theta(n^2)$ no melhor caso, caso médio e pior caso.

Assim, os resultados experimentais apresentam comportamento compatível com a análise de complexidade do algoritmo Selection Sort.

5.4 Shell Sort

Resultados dos experimentos — Shell Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	0.0	0.0	0.0
100.0	1e-06	2e-06	5e-06
1000.0	1.2e-05	2e-05	7.8e-05
10000.0	0.000171	0.000275	0.001253
100000.0	0.002235	0.003338	0.016872
1000000.0	0.02662	0.041579	0.214588

Figura 7: Tabela de tempo de execução do algoritmo Shell Sort

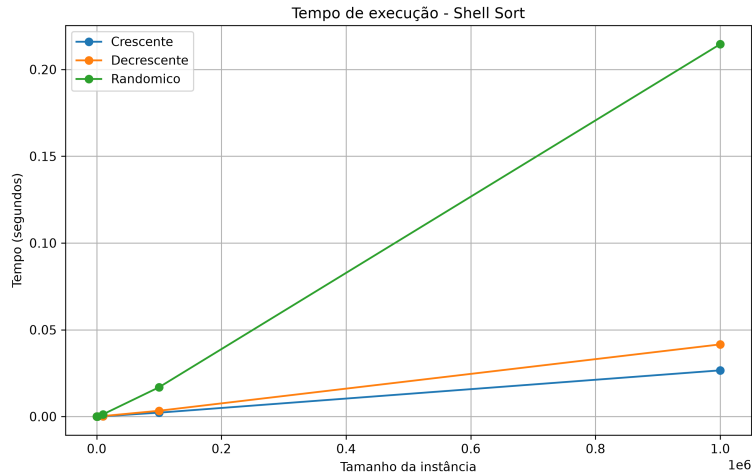


Figura 8: Gráfico de tempo de execução do algoritmo Shell Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. O algoritmo Shell Sort apresenta um comportamento diferente dos demais algoritmos analisados, pois seu desempenho é influenciado pela sequência de *gaps* utilizada. Nos experimentos deste trabalho, foi utilizado o divisor 2, produzindo uma sequência de *gaps* obtida pela divisão sucessiva do tamanho da entrada por 2 até atingir o valor 1.

Nas instâncias ordenadas de forma crescente, os elementos já se encontram em suas posições relativas para os diferentes *gaps*, reduzindo a quantidade de deslocamentos necessários durante a execução.

Nas instâncias ordenadas de forma decrescente, são necessários mais deslocamentos para reorganizar os elementos. Entretanto, os *gaps* maiores permitem que elementos que estão distantes sejam movimentados antes da etapa final, reduzindo parte da desordem do vetor.

As instâncias aleatórias apresentam comportamento intermediário, pois a quantidade de comparações e deslocamentos depende da disposição inicial dos elementos. O uso dos diferentes *gaps* permite realizar movimentações entre elementos distantes antes da ordenação final com *gap* igual a 1.

A partir dos dados, também é possível observar a influência do aumento do tamanho da entrada no tempo de execução. O crescimento observado pode ser relacionado à análise teórica do algoritmo, que depende da sequência de *gaps* utilizada. Para a sequência adotada neste trabalho, o pior caso apresenta comportamento de ordem $\Theta(n^2)$, enquanto o melhor caso apresenta comportamento de ordem $\Theta(n \log n)$.

Os resultados experimentais permitem, portanto, relacionar o comportamento obser-

vado na tabela e no gráfico com a estratégia utilizada pelo Shell Sort e com a influência da sequência de *gaps* em seu desempenho.

6 COMPARAÇÃO ENTRE OS ALGORITMOS

Além da análise individual de cada algoritmo, foram realizados experimentos comparativos entre o Insertion Sort, Bubble Sort, Selection Sort e Shell Sort, considerando os três tipos de entrada utilizados nos experimentos: crescente, decrescente e aleatória.

6.1 Entrada crescente

A Figura 9 apresenta a comparação entre os quatro algoritmos para entradas inicialmente ordenadas de forma crescente.

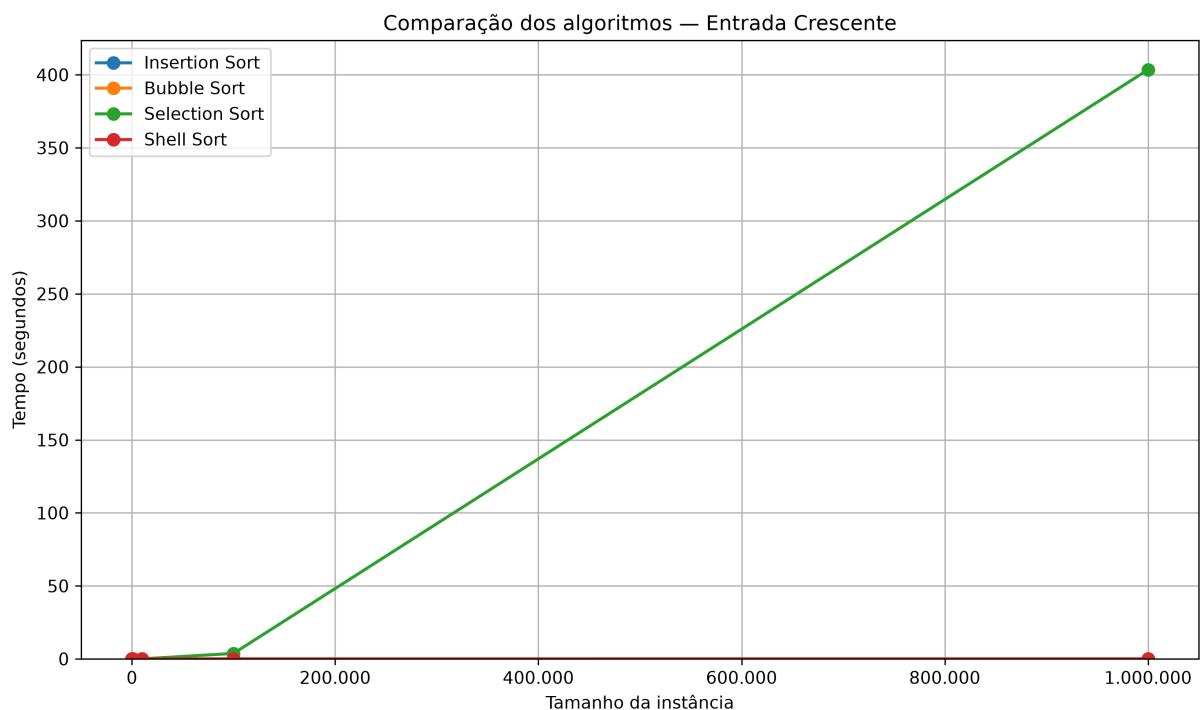


Figura 9: Comparação dos algoritmos para entrada crescente.

Observa-se que o Insertion Sort e o Bubble Sort apresentam tempos reduzidos para entradas crescentes, enquanto o Selection Sort mantém um tempo de execução significativamente maior. O Shell Sort também apresenta baixo tempo de execução para esse tipo de entrada.

6.2 Entrada decrescente

A Figura 10 apresenta a comparação entre os quatro algoritmos para entradas inicialmente organizadas em ordem decrescente.

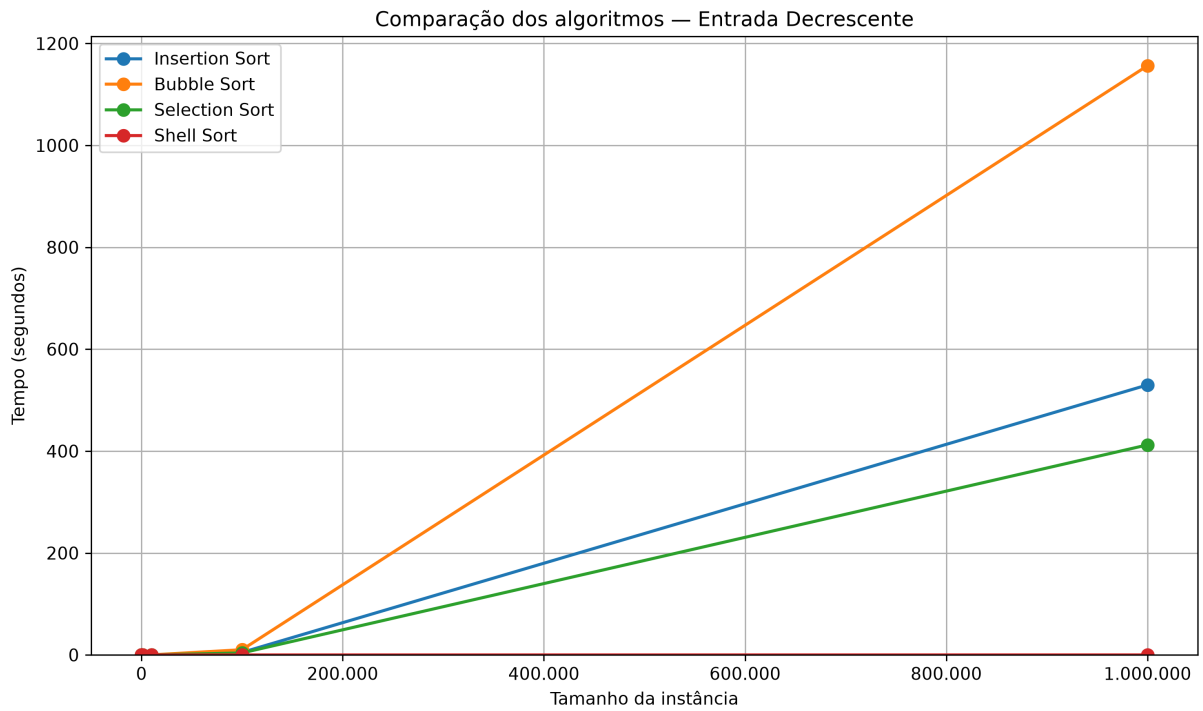


Figura 10: Comparação dos algoritmos para entrada decrescente.

Nesse caso, observa-se um aumento significativo no tempo de execução do Insertion Sort e do Bubble Sort. O Selection Sort apresenta comportamento semelhante ao observado na entrada crescente, enquanto o Shell Sort permanece com tempo de execução reduzido em relação aos demais algoritmos.

6.3 Entrada aleatória

A Figura 11 apresenta a comparação entre os quatro algoritmos para entradas geradas aleatoriamente.

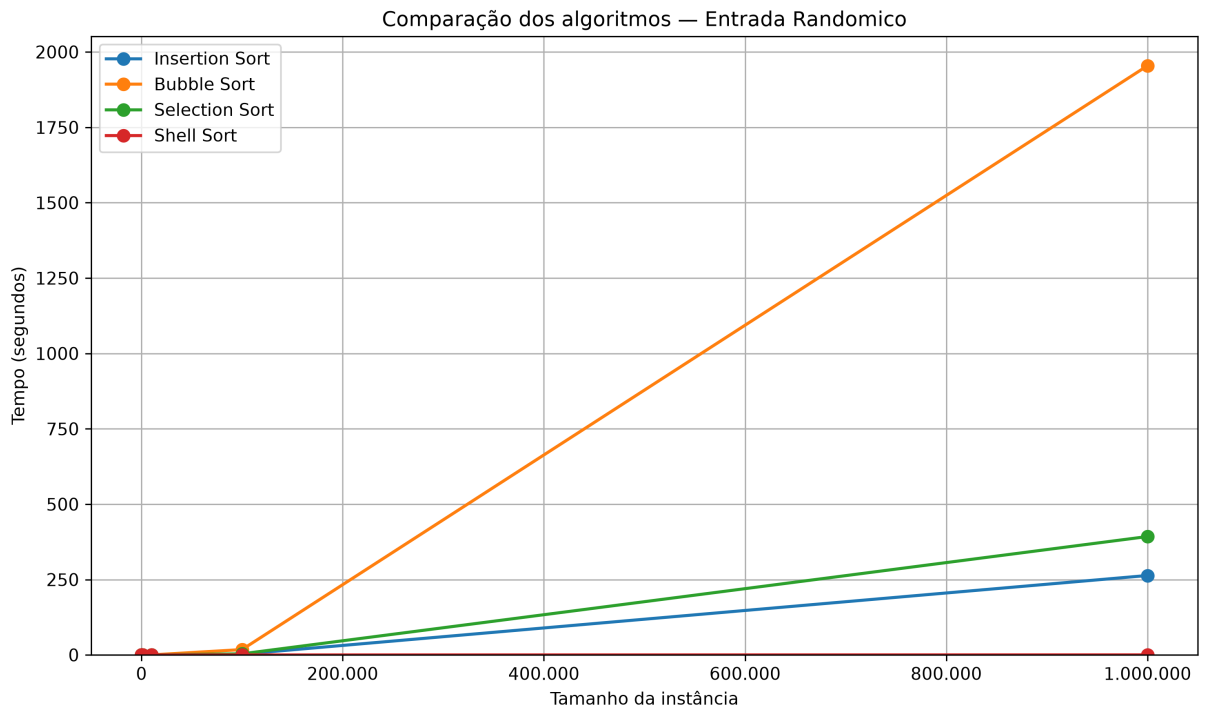


Figura 11: Comparação dos algoritmos para entrada aleatória.

Para as entradas aleatórias, observa-se uma diferença ainda maior entre os tempos de execução. O Bubble Sort apresenta o maior tempo entre os algoritmos analisados, seguido pelo Selection Sort e pelo Insertion Sort. O Shell Sort apresenta tempo significativamente menor nas instâncias utilizadas nos experimentos.

7 ATIVIDADES MANUSCRITAS

7.1 Insertion Sort

D S T Q Q S S | |
D L M M J V S

Teste de mesa manual - Insertion Sort

$V = [7, 3, 9, 2, 5]$

- $n = 5$, logo, será executado $n-1$ vezes, ou seja, 4 vezes
- Chaves $\rightarrow 3, 9, 2, 5$

Tabela: qtd. de testes de condição

Iteração	Chave	Vet. após	t_i	Deslocamentos
$i = 2$	3	[3, 7, 9, 2, 5]	2	1
$i = 3$	9	[3, 7, 9, 2, 5]	1	0
$i = 4$	2	[2, 3, 7, 9, 5]	4	3
$i = 5$	5	[2, 3, 5, 7, 9]	3	2

Soma de execuções do "while":

$$\sum t_i = 2 + 1 + 4 + 3 = 10$$

Soma dos deslocamentos:

$$\sum (t_i - 1) = 1 + 0 + 3 + 2 = 6$$

Tabela de custos:

Custo	Operação	Vezez	Tempo total
C1	inicializar 'i'	1	$O + (n)$ pode ser dado por: $T(n) = c_1 + 5c_2 + 4c_3 + 4c_4 + 10c_5 + 6c_6 + 6c_7 + 4c_8$ $T(n) = (c_1 + 5c_2 + 4c_3 + 4c_4) + n(10c_5 + 6c_6 + 6c_7 + 4c_8)$ Obs.: m. melhor caso, c6 e c7 é 0 e c5 é (n-1) pois há apenas 1 teste do 'while'.
C2	teste do 'for'	$n = 5$	
C3	atribuir 'chave'	$n - 1 = 4$	
C4	atribuir 'j'	$n - 1 = 4$	
C5	teste do 'while'	$\sum t_i = 10$	
C6	deslocamento ' $A[j+1] = A[j]$ '	$\sum (t_i - 1) = 6$	
C7	decremento 'j'	$\sum (t_i - 1) = 6$	
C8	insere 'chave'	$n - 1 = 4$	

□
□

Jandaia

Figura 12: Teste de mesa manual do algoritmo Insertion Sort

| | D S T Q Q S S
D L M M J V S

Análise de complexidade - Insertion Sort

* Considerando o mesmo vetor do teste de mesa, porém em ordenações diferentes.

1. Melhor caso:

$v = [2, 3, 5, 7, 8]$ (Já ordenado)

Custo	Vegetes	
C_1	1	$T(n) = C_1 + C_2 \cdot n + C_3(n-1) + C_4(n-1) + C_5(n-1) + C_8(n-1)$
C_2	n	
C_3	$n-1$	
C_4	$n-1$	
C_5	$n-1$	
C_6	0	Agrupando termos $\rightarrow A_n - B_n$ logo: $O(n)$
C_7	0	
C_8	$n-1$	

2. Pior caso:

$v = [9, 7, 5, 3, 2]$

Deslocamentos = $1 + 2 + 3 + \dots + (n-1)$ cada elemento vai ser deslocado até o início da parte ordenada

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \rightarrow \frac{n(n-1)}{2} \approx \frac{n^2 - n}{2}, \text{ logo: } \boxed{O(n^2)}$$

3. Caso médio:

$v = [5, 2, 7, 3, 8]$ Aqui, cada elemento precisa, em média, ser deslocado aproximadamente metade das posições possíveis.

Logo $T(n) \approx \frac{1}{2}(1 + 2 + 3 + \dots + (n-1))$

$$T(n) \approx \frac{1}{2} \cdot \frac{n(n-1)}{2} \rightarrow \frac{1}{2} \cdot \frac{n(n-1)}{2} \approx \frac{n^2 - n}{4}, \text{ logo: } \boxed{O(n^2)}$$

□

Figura 13: Análise de complexidade do algoritmo Insertion Sort

7.2 Bubble Sort

Leonardo Padovani Mangato - 9862

D S T Q Q S S | |
D L M M J V S

Teste de mesa manual - Bubble Sort

$v = [7, 3, 9, 2, 5]$

$n = 5$, loop, o loop externo será executado $(n-1)$ vezes (4x).

Tabela:

Iteração	i	Comparações	Vet. após	Trocas	bol 'trocou'
1ª passagem	0	4	[3, 7, 2, 5, 9]	3	true
2ª Passagem	1	3	[3, 2, 5, 7, 9]	2	true
3ª Passagem	2	2	[2, 3, 5, 7, 9]	1	true
4ª Passagem	3	1	[2, 3, 5, 7, 9]	0	false

Portanto:

$\Sigma \text{comparações} = 4 + 3 + 2 + 1 = 10$

$\Sigma \text{trocas} = 3 + 2 + 1 + 0 = 6$

Tabela de custos:

Custo	Operação	Veze
C1	$j = 0$	1
C2	teste "for" externo	$n = 5$
C3	$\text{trocou} = \text{false}$	$n - 1 = 4$
C4	$j = 0$	$n - 1 = 4$
C5	teste "for" interno	$5 + 4 + 3 + 2 = 14$
C6	$v[j] > v[j+1]$	10 comp.
C7	$\text{temp} = v[j]$	6
C8	$v[j+1] = v[j]$	6
C9	$v[j] = \text{temp}$	6
C10	$\text{trocou} = \text{true}$	16
C11	$j++$	10 comp.
C12	$\text{if}(!\text{trocou})$	4
C13	$i++$	4

Levando em conta o vetor: [7, 3, 9, 2, 5] (caso médio)

Aqui vai depender de cada caso, pois cada um tem um número de trocas, passagens e comparações

Jandaia

Figura 14: Teste de mesa manual do algoritmo Bubble Sort

Análise de complexidade - Bubble Sort

Considerando o mesmo vetor de teste de antes, porém, com ordenações diferentes.

1. Melhor caso:

$v = [2, 3, 5, 7, 9]$ (já ordenado)

Na primeira passagem ($i = 0$)

são feitas $n-1$ comparações, como já está ordenado, nenhuma troca acontece

"bool trocou" continua "false" e então executa o 'break'. Assim:

$$T(n) \cong c_1 + c_2 n + c_3(n-1) + c_4(n-1) + c_5(n-1) + c_6$$

como o termo de maior crescimento é $(n-1)$

$$T(n) = O(n)$$

2. Pior caso:

$v = [9, 7, 5, 3, 2]$

Como nesse caso sempre ocorrem trocas, o bloco do `if (!trocou)` nunca é executado. Assim:

$$\sum \text{comparações} = (n-1) + (n-2) + \dots + 1$$

$$\frac{n(n-1)}{2} = O\left(\frac{n^2 - n}{2}\right) \Rightarrow T(n) = O(n^2)$$

3. Caso médio:

$v = [5, 2, 7, 3, 8]$

Aqui, normalmente ocorrerão várias trocas, fazendo várias passagens, logo:

$$T(n) \cong (n-1) + (n-2) + \dots + 1$$

$$\text{Jandaia } T(n) = \frac{n(n-1)}{2} = O\left(\frac{n^2 - n}{2}\right) \Rightarrow T(n) = O(n^2)$$

Figura 15: Análise de complexidade do algoritmo Bubble Sort

7.3 Selection Sort

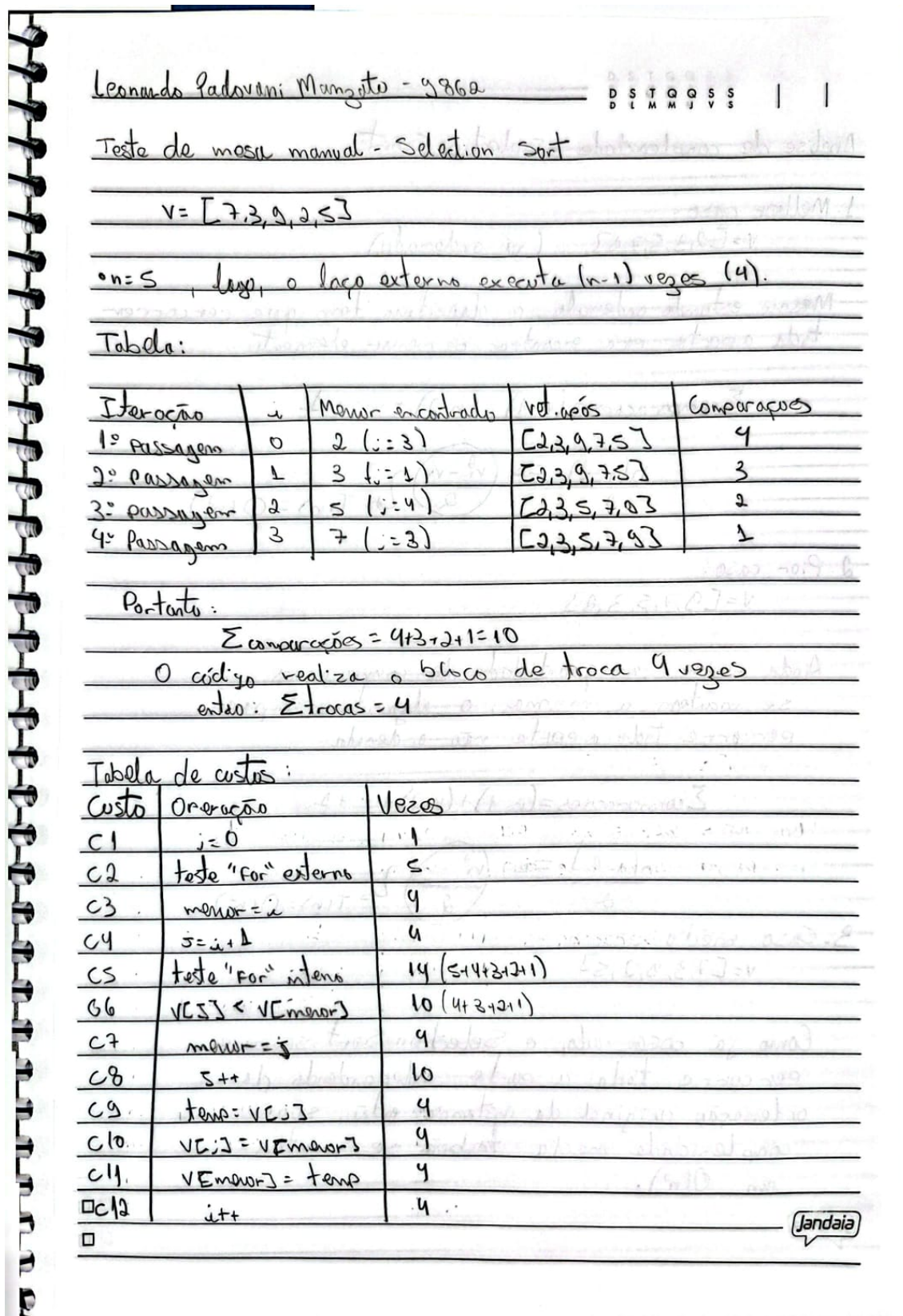


Figura 16: Teste de mesa manual do algoritmo Selection Sort

D S T Q O S S
D I M M J V S

Análise de complexidade - Selection Sort

1. Melhor caso:

$V = [2, 3, 5, 7, 8]$ (já ordenado)

Mesmo estando ordenado, o algoritmo tem que percorrer toda a parte para encontrar o menor elemento

$$\sum \text{comparações} = (n-1) + (n-2) + \dots + 1$$

$$\frac{n(n-1)}{2} = \frac{n^2 - n}{2} \Rightarrow T(n) = O(n^2)$$

2. Pior caso:

$V = [9, 7, 5, 3, 2]$

Ainda assim, a quantidade de comparações se mantém a mesma, o algoritmo sempre percorre toda a parte não ordenada.

$$\sum \text{comparações} = (n-1) + (n-2) + \dots + 1$$

$$\frac{n(n-1)}{2} = \frac{n^2 - n}{2} \Rightarrow T(n) = O(n^2)$$

3. Caso médio:

$V = [7, 3, 8, 2, 5]$

Como já observado, o Selection Sort sempre percorre toda a parte, independentemente da ordenação original da instância, então, sua complexidade média também se mantém em $O(n^2)$.

Jandaia

Figura 17: Análise de complexidade do algoritmo Selection Sort

7.4 Shell Sort

Leonardo Palovani Mangato - 9862

Teste de mesa manual - Shell Sort

$V = [7, 3, 9, 2, 5]$

- $n = 5$
- divisor $\log_{\text{gap}} = 2$
- $\text{gap} = n/2 = 2$
- $\text{gap} = 2/2 = 1$

Tabela:

Gap	i	temp	Val. após	Deslocamentos
2	2	9	[7, 3, 9, 2, 5]	0
2	3	2	[7, 2, 9, 3, 5]	1
2	4	5	[5, 2, 7, 3, 9]	2
1	1	2	[2, 5, 7, 3, 9]	1
1	2	7	[2, 5, 7, 3, 9]	0
1	3	3	[2, 3, 5, 7, 9]	2
1	4	9	[2, 3, 5, 7, 9]	0
/ 13				0/6

Jandaia

Figura 18: Teste de mesa manual do algoritmo Shell Sort

D S T Q Q S S
 D I M M J V S

Análise de complexidade - Shell Sort

1. Melhor caso:
 $V = [2, 3, 5, 7, 9]$ (já ordenado)

$n = 5$; $gap = n/2 = 2$ } $gaps = 2, 1$
 $gap = 2/2 = 1$ }
 $etapas \approx \log_2(n)$

$T(n) \approx n \cdot \log_2(n)$
 $T(n) = O(n \log_2(n))$

2. Pior caso:
 $V = [9, 7, 5, 3, 2]$

Quando o $gap = 1$, o algoritmo age semelhante ao Insertion Sort.

$T(n) = 1 + 2 + \dots + (n-1)$
 $T(n) = (n(n-1)) \div 2$
 $T(n) = (n^2 - n) \div 2$
 $T(n) = O(n^2)$

3. Caso médio:
 $V = [7, 3, 9, 2, 5]$

O custo depende da sequência de gaps e da distribuição dos elementos

Nesse trabalho:
 $gap = n/2, n/4, n/8, \dots, 1$

Jandaia
 Portanto não será atribuída uma única complexidade

Figura 19: Análise de complexidade do algoritmo Shell Sort

8 CONCLUSÃO

A realização deste trabalho permitiu compreender o funcionamento dos algoritmos Insertion Sort, Bubble Sort, Selection Sort e Shell Sort, além de analisar seus comportamentos em diferentes tipos de entrada. Por meio das implementações, dos testes de mesa, das análises de complexidade e dos experimentos práticos, foi possível observar as diferenças entre as estratégias utilizadas por cada algoritmo.

A análise teórica mostrou que o Insertion Sort apresenta complexidade $\Theta(n)$ no melhor caso, quando o vetor já está ordenado, e $\Theta(n^2)$ no caso médio e no pior caso. O Bubble Sort e o Selection Sort, considerando as implementações utilizadas neste trabalho, apresentam complexidade $\Theta(n^2)$ nos três casos analisados. Já o Shell Sort possui comportamento dependente da sequência de *gaps* utilizada. Na implementação estudada, utilizando a divisão sucessiva do *gap* por 2, o melhor caso apresenta crescimento de ordem $\Theta(n \log n)$, enquanto o pior caso apresenta complexidade $\Theta(n^2)$.

Os experimentos realizados com entradas crescentes, decrescentes e aleatórias permitiram observar na prática as diferenças previstas pela análise teórica. No Insertion Sort, a organização inicial dos elementos possui influência significativa no tempo de execução, principalmente quando o vetor já está ordenado ou quando está organizado de forma decrescente. No Bubble Sort e no Selection Sort, a quantidade de comparações permanece elevada mesmo quando a entrada já está ordenada. No Shell Sort, o uso de *gaps* permite realizar movimentações entre elementos mais distantes antes da etapa final com *gap* igual a 1.

Também foi possível observar que o aumento do tamanho da entrada provoca um crescimento no tempo de execução dos algoritmos. Esse crescimento se torna mais significativo nos algoritmos e situações em que há maior quantidade de comparações, trocas ou deslocamentos. Dessa forma, os resultados experimentais possibilitam relacionar o comportamento observado nos gráficos e tabelas com as complexidades obtidas na análise teórica.

No caso do Shell Sort, verificou-se ainda que a escolha do divisor possui influência na sequência de *gaps* utilizada. Um divisor menor produz uma quantidade maior de etapas, enquanto um divisor maior reduz essa quantidade. Entretanto, a quantidade de etapas, por si só, não determina o tempo de execução, pois a sequência de *gaps* também influencia a quantidade de elementos que precisam ser comparados e deslocados. Nos experimentos

realizados neste trabalho, foi utilizado o divisor 2 como padrão.

Dessa forma, o trabalho permitiu relacionar conceitos teóricos de análise de algoritmos com resultados obtidos por meio de experimentos práticos. A comparação entre os quatro algoritmos mostrou que, apesar de todos possuírem o mesmo objetivo de ordenar os elementos, suas estratégias de execução produzem comportamentos diferentes de acordo com o tamanho e a organização inicial da entrada.

9 REFERÊNCIAS BIBLIOGRÁFICAS

Referências

- [1] DEVFURIA. *Introdução ao algoritmo de ordenação Insertion Sort*. Disponível em: <http://devfuria.com.br/logica-de-programacao/introducao-ao-algoritmo-de-ordenacao-insertion-sort/>. Acesso em: 14 set. 2026.
- [2] CPLUSPLUS.COM. *rand - C++ Reference*. Disponível em: <https://cplusplus.com/reference/cstdlib/rand/>. Acesso em: 14 set. 2026.
- [3] GURU99. *C++ Manipulação de arquivos: como abrir, gravar, ler e fechar arquivos em C++*. Disponível em: <https://www.guru99.com/pt/cpp-file-read-write-open.html>. Acesso em: 14 set. 2026.
- [4] NGHIIATU. *8 maneiras de medir o tempo de execução em C / C++*. Disponível em: <https://ichi.nghiatu.com/pt/8-maneiras-de-medir-o-tempo-de-execucao-em-c-c-79591185633529>. Acesso em: 14 set. 2026.
- [5] STACK OVERFLOW EM PORTUGUÊS. *Como criar parâmetros opcionais em C++?*. Disponível em: <https://pt.stackoverflow.com/questions/514913/como-criar-par%3a2metros-opcionais-em-c>. Acesso em: 27 set. 2026.
- [6] SCRIBD. *Método de Shell Sort Explicado*. Disponível em: <https://pt.scribd.com/document/945722840/Shell-Sort>. Acesso em: 27 set. 2026.
- [7] TRYBE. *Bubble Sort: o que é e como usar? Exemplos práticos!*. Disponível em: <https://blog.betrybe.com/tecnologia/bubble-sort-tudo-sobre/>. Acesso em: 27 set. 2026.
- [8] JOÃO ARTHUR BARBOSA MOURA. *Estruturas de Dados e Algoritmos*. Disponível em: <https://joaoarthurbm.github.io/eda/posts/selection-sort/>. Acesso em: 27 set. 2026.

- [9] GEEKSFORGEEKS. *Comparison among Bubble Sort, Selection Sort and Insertion Sort*. Disponível em: <https://www.geeksforgeeks.org/dsa/comparison-among-bubble-sort-selection-sort-and-insertion-sort/>. Acesso em: 27 set. 2026.
- [10] EDUCBA. *Selection sort vs Bubble sort / Differences of Selection sort vs Bubble sort*. Disponível em: <https://www.educba.com/selection-sort-vs-bubble-sort/>. Acesso em: 27 set. 2026.
- [11] STACK OVERFLOW EM PORTUGUÊS. *Como passar uma função como parâmetro em C?*. Disponível em: <https://pt.stackoverflow.com/questions/2983/como-passar-uma-fun%C3%A7%C3%A3o-como-par%C3%A2metro-em-c>. Acesso em: 14 set. 2026.
- [12] VISUALGO. *Sorting*. Disponível em: <https://visualgo.net/en/sorting>. Acesso em: 27 set. 2026.
- [13] PROGRAMIZ. *Sorting Algorithms*. Disponível em: <https://www.programiz.com/dsa/sorting-algorithm>. Acesso em: 27 set. 2026.
- [14] PROGRAMIZ. *Shell Sort*. Disponível em: <https://www.programiz.com/dsa/shell-sort>. Acesso em: 27 set. 2026.
- [15] OPENAI. *ChatGPT*: ferramenta de inteligência artificial utilizada como auxílio na elaboração e revisão dos scripts de geração de gráficos e tabelas. 2026.

10 FUNÇÃO PARA CALCULAR O TEMPO

Listing 5: Função utilizada para medir o tempo de execução

```
1 double calcularTempo(void (*algoritmo)(int*, int), int* vetor, int
  tamanho) {
2     // salvar tempo do inicio
3     auto inicio = chrono::steady_clock::now();
4
5     // executar o algoritmo
6     algoritmo(vetor, tamanho);
7
8     // salvar o tempo do fim da execucao
9     auto fim = chrono::steady_clock::now();
10
11    // calcular duracao
12    chrono::duration<double> tempo = fim - inicio;
13
14    return tempo.count();
15 }
```

A função apresentada no Código 5 é responsável por medir o tempo de execução do algoritmo. Para isso, registra o instante inicial, executa o algoritmo, registra o instante final e calcula a diferença entre os dois instantes.

O valor retornado pela função representa o tempo de execução em segundos.